

PREMISE SELECTION IS NOT ENOUGH: TYPED PROOF-ACTION PORTFOLIOS FOR LEAN

Anonymous authors

Paper under double-blind review

ABSTRACT

Premise selection is usually treated as the problem of choosing which lemmas to send to a Lean hammer. Our experiments show that this view is incomplete: the same selected names can succeed or fail depending on whether they are exposed as facts, simp lemmas, HammerCore inputs, or Aesop rules. We study *typed proof-action intervention*, where a method chooses both evidence and the Lean interface through which that evidence reaches proof search. We introduce TAP-HAMMER, a compute-budgeted proof-action portfolio over replayable Mathlib theorem contexts, and evaluate it in a Mathlib 4.30 + LeanHammer 4.30 pre-theorem replay harness. On 230 replayable traced-corpus goals, the typed action grid reaches 58/230 kernel-verified proofs versus 38/230 for the best single action; a four-retry fixed portfolio reaches 57/230 out of fold and 58/230 when train-fitted. The mechanism is counterintuitive: Aesop with selected names exposed jointly as facts and simp lemmas solves 38/230 goals, while facts-only solves 5/230, simps-only solves 4/230, and broader 32-name exposure drops sharply. We also report trace-core and bridge experiments showing why failure feedback and premise budgets are useful discovery signals, but the supported main claim is sharper: for bounded Lean proof search, premise selection must be typed by proof action, not merely expanded.

1 INTRODUCTION

Lean hammers and neural theorem provers depend on premise selection: given a proof goal, the system must choose accessible lemmas that make proof search tractable and reconstructable in Lean. Modern systems therefore focus on retrieving better premises before running a backend prover (Yang et al., 2023; Zhu et al., 2025; Gao et al., 2026). This framing hides a second decision that is just as important in Lean: once a name is selected, it must enter a concrete proof interface. A name can be used as a Hammer premise, a HammerCore fact, a simp lemma, an Aesop safe rule, or part of a tactic-specific local context. These interfaces are not interchangeable.

The central challenge is that useful evidence is not monotonic under either quantity or interface. Adding more names can improve proof-core coverage, but it can also increase search branching, timeout risk, and reconstruction ambiguity. More subtly, exposing a name through the wrong tactic interface can make it inert or harmful even when the name itself is relevant. Our Mathlib experiments make this visible: Aesop with selected names exposed jointly as facts and simp lemmas proves 38/230 replayable traced-corpus goals, while facts-only exposure proves 5/230 and simps-only exposure proves 4/230. Increasing the selected Aesop pool to 32 names also hurts sharply. The key observation is therefore not simply that “more premises can hurt”, but that premise evidence must be typed by the proof action that consumes it.

We propose TAP-HAMMER, a typed proof-action portfolio for bounded Lean proof search. Given a replayable theorem context and a candidate set, TAP-HAMMER constructs a small set of proof actions that expose selected names through different Lean mechanisms: Hammer, HammerCore, simplification tactics, `solve_by_elim`, and Aesop variants that distinguish fact and simp channels. The method is deliberately compute-budgeted: every comparison starts from the same empty-premise Hammer attempt and then spends a fixed number of retry actions. This makes the main question reviewer-checkable: how much of the verified oracle headroom can a small typed portfolio recover, and which interfaces explain the gains?

The main verified result is on non-generated replayable Mathlib theorem contexts. We migrate traced goals to a Mathlib 4.30 + LeanHammer 4.30 pre-theorem replay harness and identify 230 replayable goals from 490 cleaned traces. On this subset, the typed action grid reaches a 58/230 oracle, compared with 38/230 for the best single action. A fixed portfolio of four typed retry actions after `hammer_empty` reaches 57/230 out of fold and 58/230 when train-fitted. The strict action-dependent subset contains 29 goals where the empty-premise attempt fails but some non-empty or non-default typed action succeeds; Aesop covers 20 of these goals and is the only successful family for 6.

This paper makes three contributions. First, we formulate typed proof-action intervention as a distinct problem from premise retrieval: selected evidence must be routed through a Lean interface, not only ranked. Second, we implement and evaluate TAP-HAMMER in a real Mathlib 4.30 + LeanHammer 4.30 replay harness, showing that small typed portfolios recover nearly all current oracle headroom on the replayable subset. Third, we provide mechanism evidence and boundaries: Aesop facts+sims exposure is qualitatively different from facts-only or sims-only exposure, broad 32-name insertion is harmful, naive rewrite expansion has 0/288 verified successes, and low-capacity adaptive routing does not yet beat the fixed typed portfolio. Trace-core failure-feedback experiments remain important discovery evidence, but the current supported main claim is typed proof-action exposure.

The scope is deliberately narrower than a full theorem-proving leaderboard. We treat fixed typed portfolios as strong matched-compute controls, not as weak baselines, and we do not claim adaptive routing until it beats them. We also distinguish controlled lexical retrieval proxies from full LeanSearch v2 reproductions, and bridge replay from open-ended proof generation. These boundaries keep the paper’s claim reviewer-checkable: the contribution is a typed interface mechanism for selected evidence in Lean proof search.

2 RELATED WORK

Premise selection for Lean. Premise selection is a long-standing interface between theorem provers and large formal libraries. LeanDojo introduced infrastructure and benchmarks for retrieval-augmented theorem proving in Lean (Yang et al., 2023). Recent work on Lean hammers studies learned premise ranking and ATP integration for Lean (Zhu et al., 2025), while LeanSearch v2 focuses on global premise retrieval in Lean 4 (Gao et al., 2026). These works motivate our setting and provide the strongest conceptual baselines: retrieve useful premises from a large library before proof search. Our distinction is that TAP-HAMMER studies the next interface decision after retrieval. It asks whether selected names should be exposed as Hammer facts, HammerCore inputs, simp lemmas, Aesop rules, or tactic-specific local evidence.

Proof repair, compiler feedback, and agentic provers. Several recent systems use Lean feedback to repair or generate proof text. APRIL learns to repair Lean proofs from compiler feedback (Wang et al., 2026); APOLLO and OProver frame formal proving as a broader agentic collaboration or whole-proof search problem (Ospanov et al., 2025; Ma et al., 2026). Process-verified reinforcement learning also uses Lean verification as a training signal (Kim & Yun, 2026). TAP-HAMMER is complementary to these systems: it does not edit proof text or search over natural-language proof plans. Its object is the typed action by which selected names are supplied to Lean’s existing proof mechanisms.

Search guidance and local context. Search-guidance methods such as LeanProgress predict proof progress to steer theorem proving (Huang et al., 2025), while long-context theorem proving benchmarks such as miniCTX emphasize the importance of local and user-provided context (Hu et al., 2024). These lines share our view that static global retrieval is incomplete. Our experiments focus on a different mechanism: typed exposure of selected evidence under fixed retry budgets. Failure feedback remains useful in our trace-core discovery experiments, but the verified Mathlib replay result shows that interface choice itself can dominate low-capacity adaptive routing.

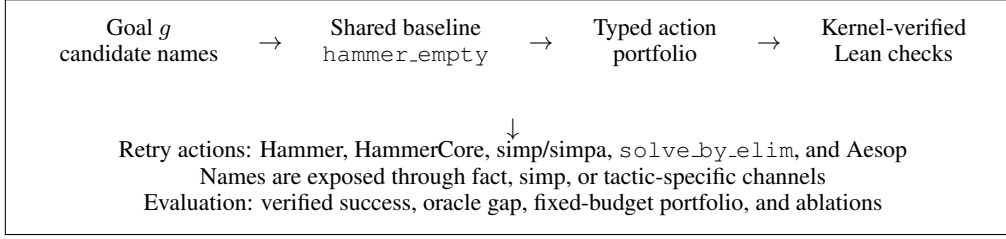


Figure 1: TAP-HAMMER evaluates typed evidence exposure under a fixed retry budget. The same selected names can be routed through different Lean interfaces, and each routed action is checked by the Lean kernel.

3 PROBLEM FORMULATION

Typed proof-action search under a fixed retry budget. Let g be a Lean goal and let $\mathcal{A}(g)$ be the accessible name set induced by imports, namespaces, declarations, and local context. A typed proof action is a pair $a = (\tau, P)$, where τ is a Lean interface such as Hammer, HammerCore, simplification, `solve_by_elim`, or Aesop, and $P \subset \mathcal{A}(g)$ is the set of names exposed through that interface. A proof-action selector chooses a sequence

$$a_t = (\tau_t, P_t), \quad |P_t| \leq k_t,$$

and runs each action under a search budget B_t . The prover returns either a kernel-verified success or a failure observation F_t , such as a timeout, missing premise signal, type mismatch, typeclass failure, failed simplification, or reconstruction failure. A closed-loop selector is a policy

$$a_{t+1} = \pi(g, a_{\leq t}, F_t, H_t),$$

where H_t includes previous attempt history and candidate features. The objective is to maximize verified success under a fixed total retry budget, rather than to maximize static premise recall at a single k .

Evaluation axes. We separate three notions that are often conflated. *Kernel-verified proof-action success* means that a patched Lean file containing a candidate proof action checks successfully. This is the main verified metric for TAP-HAMMER. *Trace-core success* means that the selected premise set covers the traced proof-core premises for a goal; it is a premise-recovery metric, not a full proving metric. *Bridge verified success* is stricter than trace-core success: trace-core recovery must coincide with successful replay of the original traced tactic script in a real Lean process. Bridge verification is still not a full proof-generation benchmark, because it replays known tactic scripts, but it filters out trace-core cases that drift too far from Lean execution.

4 METHOD

4.1 OVERVIEW

TAP-HAMMER treats a selected name as incomplete until it is assigned to a proof-action interface. Figure 1 gives the full loop. The system first builds a candidate pool and runs a shared empty-premise Hammer attempt. It then spends a fixed retry budget on typed actions that expose selected names as Hammer facts, HammerCore inputs, `simp` lemmas, `solve_by_elim` facts, or Aesop facts/`simp` rules. The important design choice is that the retry budget is spent across interfaces, not across larger untyped premise lists.

4.2 TYPED PROOF-ACTION PORTFOLIO

The portfolio is built from action families that consume selected names differently. Hammer actions pass names as ordinary premises; HammerCore actions expose a smaller internal proof-search interface; simplification actions use selected names as rewrite/`simp` lemmas; `solve_by_elim` uses names as local facts; Aesop actions can add names as safe facts, `simp` rules, or both. For each

replayable theorem context, we materialize a patched pre-theorem Lean file for each action and classify the result by running Lean with a fixed timeout.

The fixed portfolio is chosen greedily on the training split and evaluated out of fold. Every policy receives the same first action, `hammer_empty`, and then a retry budget K . This protocol makes the strongest control explicit: an adaptive policy is only useful if it beats the best fixed sequence of typed actions under the same number of Lean calls. In the current matrix, the train-fitted four-action portfolio after `hammer_empty` is `aesop_core_plus_learned`, `hammerCore_core_plus_learned`, `hammer_core_plus_learned16`, and `solve_by_elim_core`.

This design also prevents a common misreading of the method. The fixed portfolio is not intended to prove that a hand-written order is optimal; it is the hard control that any learned action router must beat. Its near-oracle performance is therefore evidence that interface typing itself is a large effect, while the absence of an adaptive win is reported as a boundary rather than hidden as a negative result.

4.3 AESOP FACT/SIMP EXPOSURE

Aesop is the most important new interface in the verified matrix, so we isolate how selected names enter it. Given a ranked name list, we split available names into fact candidates and simp candidates, then instantiate Aesop in three exposure modes: facts-only, simps-only, and facts+simps. We also vary source pools: core names, learned-only names, and core+learned names at budgets 8, 16, and 32. This ablation tests whether the Aesop gain is merely due to adding more names.

The Aesop ablation is not a new oracle-improving action set; its role is explanatory. Facts+simps exposure recovers the strong 38/230 Aesop result, while facts-only and simps-only collapse to 5/230 and 4/230. The 32-name variants are also worse than the 8/16-name variants. This supports the main mechanism claim: successful proof search depends on typed exposure and budgeted interface choice, not on untyped premise expansion.

4.4 TRACE-CORE DISCOVERY CONTROLLERS

The typed-portfolio study was motivated by earlier trace-core controllers. In local Mathlib settings, FAR-HAMMER uses visible candidate features and failure types to decide how to rerank or shrink premise sets. In imported-core settings, where lexical and same-file priors are too weak, FAR-HAMMER trains a global retriever from traced proof-core labels and then trains a second-stage scorer conditional on the first failure. These controllers are not the verified main method in this draft, but they provide discovery evidence for two ideas that survive in the verified study: premise budgets are non-monotonic, and retry decisions should be evaluated against strong fixed-budget controls.

4.5 WHY TYPED ACTIONS SHOULD HELP

Let $U(p, \tau; g)$ denote the latent utility of exposing name p through interface τ for goal g . A one-shot retriever estimates a marginal score over names, while a typed action policy estimates a score closer to $P(U = 1 \mid g, p, \tau, H_t)$. If the interface carries information about utility, then typing the action can reduce uncertainty:

$$I(U; \tau \mid g, p, H_t) > 0 \quad \Rightarrow \quad H(U \mid g, p, H_t, \tau) < H(U \mid g, p, H_t).$$

This does not imply that any larger portfolio works. A typed action can still expose invalid names, send facts to the wrong channel, or exceed the search budget. The method is useful only when interface choice is informative and the portfolio is small enough to preserve bounded proof search.

5 EXPERIMENTS

5.1 PROTOCOL

We evaluate two evidence layers. The main verified layer is a Mathlib 4.30 + LeanHammer 4.30 pre-theorem replay harness. We start from 490 cleaned traced-corpus goals, keep the 230 goals whose original traced tactic script replays in the patched pre-theorem context, and then run typed proof actions in Lean. Success means that the patched Lean file checks successfully.

Table 1: Main claim-evidence map. The primary evidence is the verified typed proof-action matrix; trace-core and bridge rows are supporting discovery evidence.

Claim	Strong control	Supported result	Readout
Typed actions matter	Best single action: 38/230	Oracle typed grid: 58/230	+20 goals
Small portfolios recover headroom	Empty Hammer: 29/230	Fixed K=4 OOF: 57/230	near oracle
Aesop exposure is typed	Facts-only: 5/230; sims-only: 4/230	Facts+sims: 38/230	interface effect
Broad insertion can hurt	Core+learned8/16 group: 41/230	Core+learned32 group: 7/230	non-monotonic
Failure feedback is useful discovery evidence	Learned+base fallback: 86.3% mean	Trace-core final-base8: 92.9% mean	supporting result
Bridge replay supports premise recovery	Fallback bridge: 44.5%	Final-base8 bridge: 53.5%	supporting result

Table 2: Verified typed proof-action matrix on 230 replayable Mathlib 4.30 traced-corpus theorem contexts. Success means the patched Lean file checks.

Matrix	Attempts	Verified attempts	Best single	Oracle
Original typed grid	3450	246	31/230	51/230
Extended typed grid	7130	506	38/230	58/230
Full Aesop-ablation grid	11500	582	38/230	58/230

The second layer is trace-core discovery evidence from earlier premise-selection experiments. There, success means that the selected premises cover the proof-core premises extracted from traced proofs. We keep these results because they explain why premise budgets and failure feedback matter, but they are not the main verified claim. BM25 and iterative lexical rows are controlled retrieval policies inside our candidate-pool protocol, not full system-to-system reproductions of the published LeanSearch v2 pipeline.

5.2 VERIFIED TYPED PROOF-ACTION MATRIX

Table 2 reports the main verified experiment. The original scaled matrix already shows action-dependent headroom: 51/230 oracle versus 31/230 for the best single action. Extending the action set with Aesop and additional typed interfaces raises the oracle to 58/230 and the best single action to 38/230. The full Aesop-ablation matrix does not raise the oracle further, which means the ablation is explanatory rather than a ceiling improvement.

The strict action-dependent subset contains 29 goals where `hammer.empty` fails but some non-empty or non-default action succeeds. Aesop solves 20/29 strict goals and is the only successful family for 6, HammerCore solves 12/29 and is the only family for 5, and `solve_by_elim` contributes 5/29 with 2 only-family cases. This distribution rules out a Hammer-only interpretation and motivates a typed-interface portfolio.

5.3 BUDGETED PORTFOLIOS AND AESOP ABLATION

Table 3 evaluates fixed and adaptive policies under matched retry budgets after the shared `hammer.empty` attempt. The fixed typed portfolio is the strongest current policy: it reaches 55/230 with two retries, 57/230 with four retries out of fold, and 58/230 when the four-action sequence is fit on all training data. Naive NB/kNN routing does not beat this control, so adaptive routing is not a supported main claim.

Table 4 isolates the strongest interface. Facts+sims exposure recovers the 38/230 Aesop result, while stripping either channel collapses success. The top-32 variants are also much worse than top-8/16 variants, giving a backend-verified version of the anti-more-premises insight.

Table 3: Budgeted policy evaluation after `hammer_empty`. The fixed typed portfolio is the strongest current control; low-capacity adaptive routing does not beat it.

Retry budget K	Fixed greedy OOF	Residual NB OOF	Residual kNN OOF	Oracle
1	49/230	29/230	44/230	58/230
2	55/230	50/230	54/230	58/230
3	55/230	55/230	55/230	58/230
4	57/230	55/230	57/230	58/230
6	57/230	57/230	57/230	58/230

Table 4: Focused Aesop ablation. Aesop gains require exposing selected names through both fact and simp channels; broader insertion is not monotonic.

Aesop group	Verified goals	Interpretation
core+learned8/16, facts+sims	38/230	best Aesop action
facts-only exposure	5/230	fact channel alone fails
sims-only exposure	4/230	simp channel alone fails
source group core+learned8/16	41/230	best source coverage
source group core+learned32	7/230	broad insertion hurts

5.4 LOCAL TRACE-CORE RECOVERY AND REPLAY BRIDGE

Table 5 shows the local Mathlib result. Blind top- k expansion is already strong, but failure-conditioned visible features improve over it while using fewer average premises. The replay bridge preserves the same ordering and magnifies the practical gap: FAR-HAMMER reaches 72.0% bridge verified success, compared with 43.0% for top- k expansion and 33.0% for static visible-feature reranking.

5.5 MORE PREMISES CAN HURT UNDER TIMEOUT PRESSURE

Table 6 isolates the non-monotonic premise-budget effect. Equal-budget top- k and static visible-feature reranking both timeout frequently and underperform one-shot retrieval. Blind expansion recovers some failures but still times out on 46.1% of goals. A timeout-conditioned shrink policy improves success to 69.3% and reduces timeout rate to 23.5%. This result is important because it rules out the simplistic explanation that FAR-HAMMER merely adds more premises after failure.

5.6 IMPORTED-CORE GLOBAL RETRIEVAL

Imported-core goals expose the limit of local feature heuristics. The dataset contains 2k goals with an average of 4.26 imported proof-core premises and 188.6 imported candidates per goal. In the initial global retrieval stress test, same-file prior and visible-feature policies are near zero, and BM25 / controlled iterative lexical proxies reach only around 12% success. This negative result is useful: it shows that the local controller does not solve global retrieval by itself.

After training a global retriever from traced proof-core labels, the imported-core setting becomes a strong test of failure-conditioned control. Table 7 reports the heldout result. Learned expansion reaches 84.0%, and learned+base fallback raises the strong failure-agnostic baseline to 87.0%. The learned second-stage controller improves to 92.4% while using fewer premises. The final base guardrail slightly improves the original split and is mainly justified by its stability and bridge behavior.

5.7 SPLIT STABILITY AND REAL LEAN BRIDGE

Table 8 summarizes the final imported-core controller. Across four 500-goal splits, final-base8 has 92.9% mean success with 0.9-point standard deviation, compared with 86.3% for learned+base fallback. On the 200-goal imported-core bridge, final-base8 reaches 53.5% bridge verified success versus 44.5% for fallback. This is the strongest current proof-reconstruction-facing evidence, but it should be read as a bridge validation rather than as full proof generation.

Table 5: Local Mathlib trace-core recovery and real Lean replay bridge. The 2k trace-core table measures premise recovery; the 100-goal bridge additionally requires real Lean replay.

Method	Trace success	FFR	Avg prem.	Bridge verified
one_shot	58.0%	0.0%	28.4	10.0%
visible_feature_rerank	86.5%	0.0%	42.9	33.0%
topk_expansion	91.5%	79.9%	46.9	43.0%
rule_far_no_core_tags	95.7%	89.7%	45.1	72.0%
rule_far_full (oracle ref.)	98.3%	96.0%	42.9	79.0%

Table 6: Timeout stress test on 2k local goals. Under timeout pressure, adding premises can be worse than one-shot retrieval; a failure-conditioned shrink action is the best non-oracle policy.

Method	Success	FFR	Avg prem.	Timeout rate
one_shot	57.0%	0.0%	28.4	4.0%
topk_equal_budget	43.8%	0.0%	42.9	56.2%
topk_expansion	59.7%	6.2%	50.4	46.1%
rule_far_no_core_tags	59.7%	6.2%	42.8	46.1%
rule_far_no_core_timeout_shrink	69.3%	28.6%	42.7	23.5%
rule_far_full (oracle ref.)	100.0%	100.0%	40.1	22.5%

5.8 ABLATIONS AND FAILURE DIAGNOSIS

The ablations in Table 9 identify why the final method is shaped as a typed portfolio rather than a larger untyped premise list. The verified Aesop ablation shows that fact and simp channels interact: either channel alone is weak, while their joint exposure is strong. The same table also keeps trace-core discovery ablations showing that failure conditioning and narrow guardrails are useful signals, but these are secondary to the verified typed-interface result.

6 THEORY AND ANALYSIS

Proposition 1 (Typed interfaces are useful when they are informative). *Let U be the latent utility of exposing a selected name to a Lean proof search, and let τ be the interface used to expose it. If $I(U; \tau \mid g, p, H) > 0$, then conditioning on τ strictly reduces the uncertainty of U given the goal, selected name, and history.*

Sketch. This is the standard identity $I(U; \tau \mid g, p, H) = H(U \mid g, p, H) - H(U \mid g, p, H, \tau)$. The proposition clarifies what the verified experiments test: if interface choice matters, a typed proof-action portfolio can beat any single untyped exposure rule under the same Lean-call budget.

Proposition 2 (Premise coverage is not monotonic in premise budget). *There can exist premise sets $P_1 \subset P_2$ such that P_1 covers enough useful premises for the backend to succeed under budget B , while P_2 contains the same useful premises but causes timeout or reconstruction failure under the same budget.*

Sketch. Adding premises preserves proof-core coverage but can increase branching, generated clauses, simplification search, or reconstruction ambiguity. Under fixed time/search budget, the prover may fail on the larger set even though the smaller set contains the same proof-relevant lemmas. Tables 6 and 4 are empirical versions of this proposition: top- k expansion can hurt under timeout pressure, and top-32 Aesop exposure is worse than smaller typed exposure.

Proposition 3 (A guardrail should be narrow when the learned controller is already strong). *If a learned failure-conditioned scorer improves most retry decisions but occasionally suppresses high-base-score rescue premises, then a small late-stage base guardrail can improve bridge recovery while broad base mixing can reduce success.*

Sketch. The learned controller and the base retriever encode different biases. A broad mixture changes many rankings, including rankings already corrected by failure feedback. A narrow final guardrail only affects the tail cases where high-base premises are about to be excluded. This matches

Table 7: Imported-core heldout results. The strongest fair baseline is learned+base fallback; the learned second-stage controller is the main failure-conditioned contribution.

Method	Success	FFR	Avg prem.
bm25_expansion	12.8%	8.4%	91.5
learned_rerank	64.2%	0.0%	56.0
learned_expansion	84.0%	67.7%	57.7
learned_base_fallback	87.0%	73.8%	57.6
rule_far_learned_second_stage	92.4%	84.7%	53.9
second_stage_final_base8	92.6%	85.1%	53.9

Table 8: Final imported-core stability and bridge validation. The bridge subset is disagreement-heavy and replay-filtered; it validates the premise-recovery gain under real Lean replay but is not a full theorem-proving benchmark.

Policy	Split mean	Split std	Split FFR	Bridge trace	Bridge verified
learned_base_fallback	86.3%	1.9	70.7%	67.5%	44.5%
rule_far_learned_second_stage	92.5%	0.9	84.0%	81.0%	52.5%
second_stage_final_base8	92.9%	0.9	84.8%	81.5%	53.5%

the trace-core discovery pattern: final-base8 improves bridge verified success, while hard guardrails and broad expert/base mixing are harmful. It also motivates the verified typed-action protocol, where broad insertion is treated as a hypothesis to test rather than as a default improvement.

7 LIMITATIONS

The main limitation is evaluation scope. The verified typed-action results are kernel-checked, but they are measured on the 230-goal replayable subset of a 490-goal cleaned trace pool, not on every Mathlib theorem and not as an open-ended theorem-proving benchmark. The harness patches original files in a pre-theorem context, which is the right setting for avoiding target-theorem leakage, but it also means non-replayable migrated traces are outside the current main evaluation.

A second limitation is that the current strongest policy is a fixed typed portfolio, not a learned adaptive router. The budgeted policy experiments are useful because they prevent overclaiming: low-capacity NB/kNN routers do not beat the fixed typed portfolio under matched compute. The trace-core experiments use supervised proof-core traces and should be read as discovery evidence for premise budgets and failure feedback, not as the main verified claim. Future work should either train a stronger adaptive action policy that beats the fixed typed portfolio or expand the replayable verified corpus.

A third limitation is external comparison. Our controlled BM25 and iterative lexical rows test retrieval behavior inside our candidate-pool protocol, but they are not official reproductions of LeanSearch v2 or other full premise-selection systems. The present verified result is therefore best read as orthogonal to global retrieval: once a retriever proposes names, Lean still needs a typed interface decision for how those names enter proof search.

8 CONCLUSION

This paper argues that Lean premise selection is incomplete without typed proof-action exposure. TAP-HAMMER implements this view as a compute-budgeted portfolio over Hammer, HammerCore, simplification, `solve_by_elim`, and Aesop interfaces. On replayable Mathlib theorem contexts, the typed action grid reaches 58/230 verified proofs versus 38/230 for the best single action, and a four-retry fixed portfolio recovers 57/230 out of fold. The strongest insight is counterintuitive: Aesop succeeds when evidence is exposed jointly as facts and simp lemmas, while facts-only, simp-only, and broader 32-name exposure are much weaker. This positions typed evidence exposure as a concrete mechanism for building more reliable Lean hammers, with adaptive failure-conditioned

Table 9: Focused diagnostic results. The main verified insight is typed exposure; trace-core rows provide supporting discovery evidence for non-monotonic premise budgets and failure feedback.

Question	Evidence	Interpretation
Does Aesop need both exposure channels?	Facts+simps: 38/230; facts-only: 5/230; simps-only: 4/230	The interface type changes utility.
Does adding more Aesop names help?	Core+learned8/16 group: 41/230; core+learned32 group: 7/230	Broad insertion can hurt.
Are visible features enough statically?	Static all-feature rerank: 86.5%; failure-conditioned: 95.7%	Failure event changes feature utility.
Which visible features matter most?	Name-only FAR-HAMMER: 95.0%; full visible FAR-HAMMER: 95.7%	Namespace/name morphology is strongest.
Is failure type alone enough?	Failure-type-only: 91.5%, equal to top- k expansion	Candidate-level conditioning is required.
Does broad base guardrail help?	Hard guardrails drop bridge trace from 72.0% to 63.0/60.0%	Broad rescue damages ranking.
Does expert mixing help?	Multi-expert bridge trace drops to 62.0%; expert-16 drops final-base8	Hand-written expert mixing is unstable.
Are misses candidate-pool failures?	200-bridge inspection: 0/17 candidate-pool misses	Remaining headroom is ranking/calibration.

routing remaining the next method target rather than a claim already supported by the current verified results.

REFERENCES

- Guoxiong Gao, Zeming Sun, Jiedong Jiang, Yutong Wang, Jingda Xu, Peihao Wu, Bryan Dai, and Bin Dong. Leansearch v2: Global premise retrieval for lean 4 theorem proving. *arXiv preprint arXiv:2605.13137*, 2026. URL <https://arxiv.org/abs/2605.13137>.
- Edward J. Hu et al. minictx: Neural theorem proving with long contexts. *arXiv preprint arXiv:2408.03350*, 2024. URL <https://arxiv.org/abs/2408.03350>.
- Suozhi Huang, Peiyang Song, Robert Joseph George, and Anima Anandkumar. Leanprogress: Guiding search for neural theorem proving via proof progress prediction. *arXiv preprint arXiv:2502.17925*, 2025. URL <https://arxiv.org/abs/2502.17925>.
- Minsu Kim and Se-Young Yun. Process-verified reinforcement learning for theorem proving via lean. In *International Conference on Learning Representations*, 2026. URL <https://openreview.net/forum?id=P00k4DFaXF>.
- David Ma, Kaijing Ma, Shawn Guo, Yunfeng Shi, Enduo Zhao, Jiajun Shi, Zhaoxiang Zhang, Gavin Cheung, Jiaheng Liu, and Zili Wang. Oprover: A unified framework for agentic formal theorem proving. *arXiv preprint arXiv:2605.17283*, 2026. URL <https://arxiv.org/abs/2605.17283>.
- Azim Ospanov, Farzan Farnia, and Roozbeh Yousefzadeh. Apollo: Automated llm and lean collaboration for advanced formal reasoning. In *Advances in Neural Information Processing Systems*, 2025. URL <https://openreview.net/forum?id=fxDCgOruk0>.
- Evan Wang, Simon Chess, Daniel Lee, Siyuan Ge, Ajit Mallavarapu, and Vasily Ilin. Learning to repair lean proofs from compiler feedback. *arXiv preprint arXiv:2602.02990*, 2026. URL <https://arxiv.org/abs/2602.02990>.
- Kaiyu Yang, Aidan M. Swope, Alex Gu, Rahul Chalamala, Peiyang Song, Shixing Yu, Saad Godil, Ryan Prenger, and Anima Anandkumar. Leandojo: Theorem proving with retrieval-augmented language models. In *Advances in Neural Information Processing Systems*, 2023. URL <https://arxiv.org/abs/2306.15626>.
- Thomas Zhu, Joshua Clune, Jeremy Avigad, Albert Qiaochu Jiang, and Sean Welleck. Premise selection for a lean hammer. *arXiv preprint arXiv:2506.07477*, 2025. URL <https://arxiv.org/abs/2506.07477>.

A ADDITIONAL EXPERIMENTAL DETAILS

A.1 CANONICAL OUTPUT SOURCES

All numbers in the main tables are copied from canonical JSON or Markdown outputs in the project directory:

- Verified typed action grid: `outputs/mathlib430_pretheorem_action_matrix_scaled230_aesop_ablation_merged.md`.
- Budgeted typed portfolio: `outputs/mathlib430_budgeted_action_policy_scaled230_aesop_ablation.md`.
- Aesop fact/simp ablation: `analysis/mathlib430_aesop_ablation_scaled230.md`.
- Strict interface taxonomy: `analysis/mathlib430_strict_action_taxonomy_scaled230_aesop_ablation.md`.
- Local trace-core: `outputs/phase1_mathlib_trace_core_2000_feature_ablation_a40.json`.
- Phase 1 bridge: `outputs/phase1_reconstruction_bridge_report_100.md`.
- Timeout stress: `outputs/phase2_timeout_stress_2000_a40.json`.
- Imported-core heldout: `outputs/phase3_second_stage_controller_eval_500_a40.json` plus final guardrail deltas.
- Split stability: `outputs/phase3_split_stability_summary_4x500.md`.
- Imported bridge report: `outputs/phase3_bridge_report_200.md`.
- Imported bridge taxonomy: `outputs/phase3_bridge_failure_taxonomy_200.md`.
- Imported bridge inspection: `outputs/phase3_bridge_negative_case_inspection_200.md`.

A.2 FEATURE-GROUP ABLATION

Table 10: Feature-group ablation on 2k local trace-core goals. Static visible-feature reranking is weaker than blind expansion; failure-conditioned feature use is the important step.

Method	Success	FFR	Avg prem.
<code>topk_expansion</code>	91.5%	79.9%	46.9
<code>visible_feature_name_rerank</code>	85.3%	0.0%	42.9
<code>visible_feature_statement_rerank</code>	77.6%	0.0%	42.9
<code>visible_feature_decl_rerank</code>	78.3%	0.0%	42.9
<code>visible_feature_rerank</code>	86.5%	0.0%	42.9
<code>rule_far_no_core_decl_features</code>	92.6%	82.4%	46.2
<code>rule_far_no_core_statement_features</code>	92.8%	82.9%	46.4
<code>rule_far_no_core_name_features</code>	95.0%	88.1%	45.4
<code>rule_far_no_core_tags</code>	95.7%	89.7%	45.1

A.3 IMPORTED-CORE STRESS BEFORE LEARNED RETRIEVAL

Table 11: Initial imported-core stress test on 2k goals. This table is a diagnostic negative result: local and lexical signals are insufficient for global imported-premise recovery.

Method	Success	FFR	Avg prem.
<code>one_shot</code>	1.1%	0.0%	32.0
<code>same_file_prior_rerank</code>	2.1%	0.0%	56.0
<code>visible_feature_rerank</code>	2.4%	0.0%	56.0
<code>bm25_rerank</code>	7.0%	0.0%	56.0
<code>leansearch_iterative</code>	11.9%	8.3%	91.9
<code>bm25_expansion</code>	11.9%	8.3%	92.2
<code>rule_far_bm25</code>	12.2%	8.7%	92.2
<code>rule_far_full (oracle ref.)</code>	55.7%	55.2%	86.6

A.4 BRIDGE FAILURE TAXONOMY

The 200-goal imported bridge verifies 124/200 original traced tactic scripts. The primary replay failures are 30 unknown-identifier errors, 13 other Lean errors, 8 typeclass failures, 7 unsolved-goal failures, 6 `simp` no-progress failures, 5 tactic failures, 5 type mismatches, and 2 sorry-context failures. At the policy level, final-base8 has 17 replay-verified trace misses, compared with 19 for the original second-stage controller, 35 for learned+base fallback, and 47 for learned expansion. Negative-case inspection finds 0/17 candidate-pool misses, 2 fallback-selected core drops, 3 expansion-selected core drops, and 15 failure-conditioned rank misses. This supports the limitation that remaining headroom is ranking and calibration, not candidate generation.

A.5 GUARDRAIL AND EXPERT-MIXING ABLATIONS

The final base8 guardrail was selected because broader heuristics were negative. Hard base guardrails reduce bridge trace-core success from 72.0% to 63.0% or 60.0%. Soft base-score mixing is inert at low mixing weights and harmful at high weights. Imported-failure multi-expert routing drops bridge trace-core success to 62.0%. A larger final expert guardrail also underperforms final-base8 on the original split, fold0, and bridge subset. These results justify stopping hand-written expert-mix tuning and treating calibrated expert gating as future work.

B CLAIM-EVIDENCE SELF-REVIEW

Table 12: End-of-draft self-review. Claims in the abstract and introduction are aligned to reported evidence; unsupported stronger claims are explicitly avoided.

Dimension	Current answer	Residual risk
Contribution	Typed proof-action exposure is formulated as distinct from premise ranking, with Aesop fact/simp ablations as mechanism evidence.	A fixed portfolio can look simple unless the interface mechanism remains central.
Writing clarity	Main text now leads with verified typed actions, then uses trace-core as discovery evidence.	Method names are long; final draft may need shorter aliases.
Experiments	Results cover verified replayable Mathlib actions, budgeted portfolios, Aesop ablations, trace-core discovery, and replay bridges.	Replayable subset is 230/490, not all traces.
Evaluation	Kernel-verified action success, oracle headroom, fixed-budget policy, and interface ablations are reported.	Full open-ended theorem proving is out of scope.
Soundness	Negative ablations show broad insertion and naive adaptive routing are not enough.	Adaptive routing does not yet beat fixed typed portfolios.